

Remotely Accessible Cyber-Physical System Testbed and Open Architecture Synchrophasor Systems for Bangladesh's National Power Grid's Cyber Security and Reliability

PI: Dr. Hafiz Abdur Rahman

Cyber-Physical System (CPS) Testbed

GridPACK • HELICS • NS-3 • Node-RED

System Description, Installation, and User Guide

Remotely Accessible Cyber-Physical System Testbed and Open Architecture Synchrophasor
Systems

NSU-PGCB project (EPRC/58-2018-007-01)

Md Mazharul Islam

October 2026

Repository: github.com/mazharulmd/cps-testbed-cluster

Table of Contents

1. Introduction.....	4
1.1 Why this testbed	4
1.2 Target of the project	4
1.3 This document	4
2. Key concepts	5
3. Architecture.....	6
3.1 Overview	6
3.2 The closed loop	7
3.3 HELICS messages	7
3.4 Single server and cluster.....	8
3.5 GridPACK power flow on MPI: pf_server	8
3.6 GridPACK dynamic simulation on MPI: dsf_server	9
3.7 What uses MPI	10
4. Installation.....	10
4.1 Requirements.....	10
4.2 Layout on disk	10
4.3 Native installation on Ubuntu 24.04.....	11
4.5 Logins.....	12
4.6 Services	12
4.7 Docker	12
5. Setting up a cluster.....	13
5.1 Recommended hardware	13
5.2 Steps	13
5.3 Scheduling	13
6. Using the dashboard.....	14
6.1 Opening the dashboard.....	14
6.2 Experiments page	14
6.3 Running an experiment from the form	15
6.4 Cluster page.....	16
6.5 Scenario files	17
6.6 Uploading a grid model.....	18
6.7 Live page	18
6.8 Inside page: what the engines compute.....	20
6.9 Results page.....	27
6.10 Dynamic simulation runs.....	30

6.11 IEEE 14 console (original prototype).....	31
6.12 Node-RED editor.....	33
6.13 API and command line.....	33
7. Output files.....	34
8. Validation and results	35
8.1 Test environments	35
8.2 IEEE 118-bus on the cluster.....	35
8.3 2000-bus Texas grid	35
8.4 GridPACK on MPI: accuracy and time.....	36
8.5 Dynamic simulation	36
9. Sizing and storage	37
10. Limitations and next steps	37
11. Troubleshooting	38
Appendix A. Repository layout	38

1. Introduction

1.1 Why this testbed

Most power grids are still watched by SCADA (Supervisory Control and Data Acquisition). Remote terminal units at the substations report voltage magnitudes, power flows, and breaker states to the control center, which polls them once every few seconds. These measurements have no common time stamp. Readings from different substations are taken at slightly different moments, and they carry no phase angles. The control center, therefore, sees a delayed and blurred picture of the grid. It must estimate the state using iterative calculations and can miss fast events that occur between scans, such as voltage swings, oscillations, or the first moments of a fault.

Phasor measurement units (PMUs) close this gap. They are devices at substations that measure voltage and current phasors, magnitude and angle, about 30 times a second, each stamped with GPS time. The measurements travel over a communication network to a phasor data concentrator (PDC), which aligns them by timestamp and passes them on to the control center. From them, the control center estimates the state of the entire grid and acts when something is wrong, for example, by lowering a generator's voltage setpoint.

This faster view depends on the communication network and on the trust placed in the data, so two things can go wrong between the grid and the control center:

- Network problems: the network can delay or lose measurements.
- Attacks: an attacker can change the measurements (a false data injection, FDI) so that the control center sees a safe grid while the real one is in danger.

Such situations cannot be tried on the real grid. The testbed reproduces the grid, the network, and the control center in software, enabling their study safely and repeatedly.

1.2 Target of the project

Goal	How the testbed meets it
Run the grid simulation (GridPACK) on a cluster with MPI	GridPACK runs as a single persistent MPI job per experiment, partitioned across the MPI ranks of the cluster nodes (Section 3.5).
Couple the grid, the communication network, and the control center	GridPACK, NS-3, and the control center are HELICS federates on a common simulated clock, in a closed loop (Section 3.2).
Remotely accessible testbed	Users work in a web dashboard (Node-RED) that requires a login, accessed over a VPN or SSH tunnel (Section 6).
Users bring their own scenarios and grids	Scenario files (YAML/JSON) and grid models (MATPOWER) are uploaded on the dashboard; results and a live view are shown there (Section 6).
Run on Ubuntu servers with or without containers	One installer for Ubuntu 24.04 and a Docker image built by the same scripts (Section 4).

1.3 This document

Section 2 explains the ideas in plain language. Section 3 describes the architecture; Section 4 describes the installation; Section 5 describes the cluster setup; Section 6 describes the

dashboard with screenshots; Section 7 describes the output files; and Section 8 describes the test results. Sizing, limitations, and troubleshooting follow.

2. Key concepts

The testbed is a safe "playground" for the power grid on computers, where network problems and cyber-attacks can be tested, many computers share the heavy computation, and anyone can run experiments from a browser.

Table 1. The main terms

Term	What it is	Role in the testbed
Power grid, bus	The network of power lines; a bus is a connection point (substation).	Simulated by GridPACK on IEEE test grids (14 to 300 buses) or uploaded grids (up to 10 000 buses tested).
PMU	A measuring device that reports voltage and current phasors 30 times per second.	Placed on the buses chosen by an optimal placement; simulated inside NS-3.
PDC	Collects PMU frames from the same instant into a single measurement set.	Simulated in NS-3 with a wait window for late frames.
Control center	The operator's system that estimates the grid state and acts.	Python federate: state estimation, bad data detection, generator setpoint control.
State estimation	Computing all bus voltages from the PMU measurements.	Weighted least squares on the linear PMU model.
Bad data detection	A chi-square test that flags measurements that do not fit together.	Detects and removes falsified PMUs when it can.
False data injection (FDI)	An attacker changes measurements to hide a problem.	Simple (one-channel) and stealthy (consistent changes) attacks in NS-3; also, delay and drop attacks.
Cluster, head, node	A group of computers working together; the head coordinates, and the nodes do the work.	Head: dashboard, API, HELICS broker. Nodes: GridPACK MPI ranks, NS-3, control center.
MPI	The standard way for programs on many computers to split one large calculation.	Used by GridPACK to solve the power flow on several ranks/nodes.
GridPACK	A power-system simulation library from PNNL built for MPI.	The "real" grid: a power flow every grid step (0.5 s), or a dynamic simulation of the generators every few milliseconds.
Quasi-steady state/dynamic	Two ways to simulate the grid: a series of balanced snapshots, or the machines' motion over time.	Scenario field mode: qss (power flow) or mode: dynamic (generators, exciters, and governors integrated in steps of about 5 ms).
Frequency, rotor angle	How fast the generators turn (60 Hz when balanced) and where each rotor stands relative to the others.	Shown in dynamic runs: frequency falls when generation is lost; a growing angle spread means loss of synchronism.
NS-3	A network simulator.	Simulates the PMU links, the PDC, attacks, and the command path back to the generators.
HELICS	A co-simulation framework: a referee and post office for simulators.	Keeps GridPACK, NS-3, and the control center on one clock and passes their messages.
Federate	One simulator is taking part in a HELICS co-simulation.	Grid, network, control center, and observer.
Node-RED	A visual tool for building web dashboards and data flows.	The user interface: Experiments, Live, Results, Cluster and the IEEE 14 console.

3. Architecture

3.1 Overview

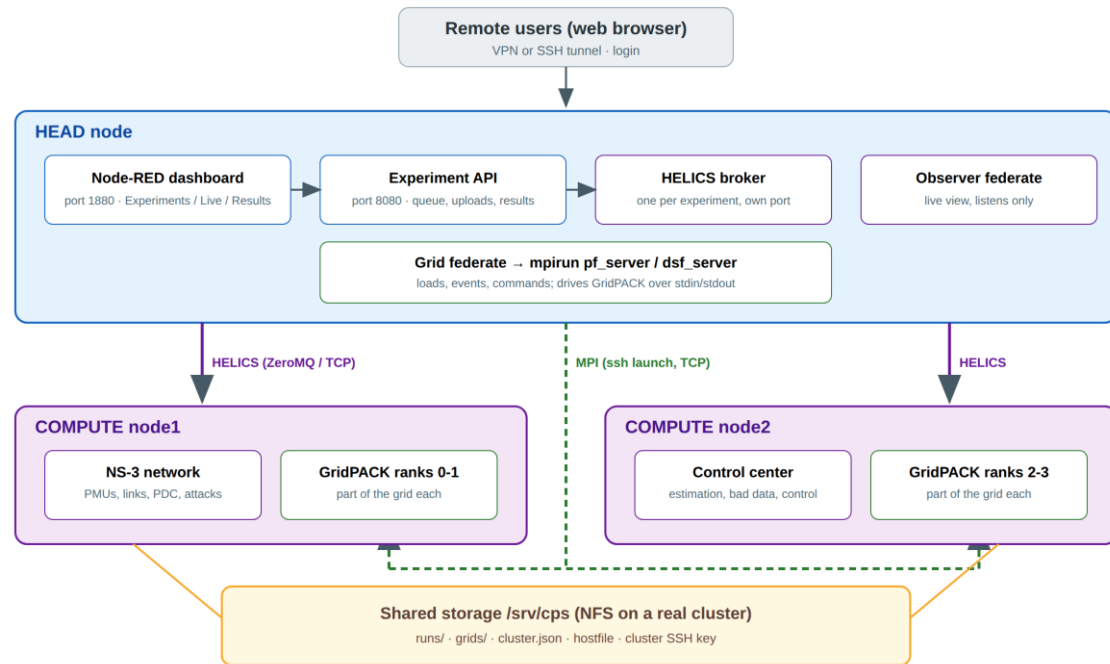


Figure 1. Architecture on a cluster with a head and two compute nodes.

A **head** node hosts the user-facing services (Node-RED dashboard and experiment API), starts one **HELICS broker** per experiment, and runs the **grid federate**, which launches GridPACK on the compute nodes. The **NS-3** and **control-center** federates can run on any node. All nodes share the data directory (via NFS on a real cluster), where results, uploaded grids, the MPI hostfile, and the cluster configuration are stored.

Table 2. Components

Component	Software	Where in the repository
Grid federate + MPI power flow or dynamic simulation	GridPACK 3.5 (patched), PETSc 3.19 (MUMPS), ParMETIS, Global Arrays 5.9, OpenMPI 4.1	federates/grid_fed.py, gridpack/pf_server/, gridpack/dsf_server/, gridpack/patches/
Network federate	NS-3.48 with the HELICS C++ library	ns3-scratch/helictest/
Control-center federate	Python (NumPy, SciPy sparse)	federates/cc_fed.py, federates/cpslib/estimation.py
Observer federate (live view)	Python	federates/observer_fed.py
Co-simulation	HELICS 3.6.1 (ZeroMQ core)	federates/run_experiment.py, federates/cpslib/cluster.py
Experiment API	FastAPI (Python)	webapp/app.py, webapp/scenario.py
Dashboard	Node-RED 4.1, FlowFuse Dashboard 2	node-red/flows.json
Installation	Bash, systemd, Docker	install/, docker/

3.2 The closed loop

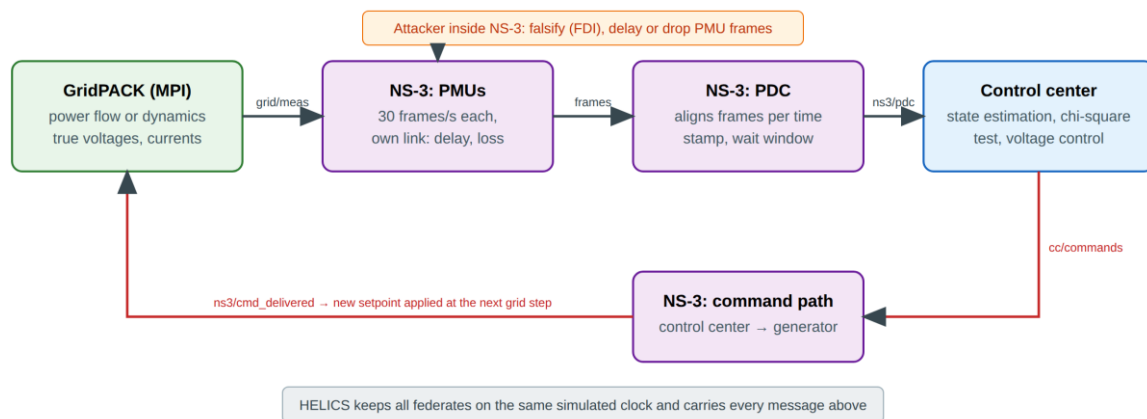


Figure 2. Data flow of one experiment. Measurements go from the grid to the control center; commands go from the control center back to the grid.

At each grid step, GridPACK solves the power flow, and the grid federate publishes the true phasors that each PMU would measure. In NS-3, every PMU sends 30 frames per second over its own link, with its own delay, jitter, and loss, to the PDC, which releases time-aligned sets to the control center. The control center estimates the state, runs the bad data test, and, when a bus voltage is out of limits, sends a setpoint command to the nearest generator. The command travels back through NS-3, and GridPACK applies it at the next grid step. An attacker inside NS-3 can falsify, delay, or drop the frames of selected PMUs.

In dynamic mode, the grid step is one PMU frame (1/30 s): GridPACK integrates the machines up to the frame time, so every frame carries the transient (voltage dips, swings, frequency changes), and a command is applied by moving the exciter reference of the generator at the next frame.

3.3 HELICS messages

Table 3. HELICS publications

Topic	From → to	Content
grid/meas	grid → NS-3	True voltage and current phasors of every PMU, each grid step
ns3/pdc	NS-3 → control center	Time-aligned PMU sets released by the PDC
cc/commands	control center → NS-3	Generator setpoint commands
ns3/cmd_delivered	NS-3 → grid	Commands that reached the generator, with delivery time
grid/status	grid → observer	All true bus voltages and the commands applied; in dynamic mode, also frequencies and rotor angles (live view only)
cc/status	control center → observer	Chi-square test, alarms, completeness, estimates (live view only)

3.4 Single server and cluster

Table 4. Where each part runs

Part	Single server	Cluster (example)
Node-RED, API, HELICS broker, observer	the server	head
Grid federate (starts mpirun)	the server	head
GridPACK MPI ranks	the server's cores	compute nodes listed in CPS_NODES (e.g. 2 ranks on node1, 2 on node2)
NS-3 federate	the server	node set by CPS_PLACE_NS3 (e.g. node1)
Control-center federate	the server	node set by CPS_PLACE_CC (e.g. node2)
Data /srv/cps	local disk	shared over NFS by all nodes

3.5 GridPACK power flow on MPI: pf_server

pf_server is a small GridPACK application written for the testbed. It is started once per experiment with `mpirun -np N` (and a hostfile on a cluster). GridPACK reads the grid and partitions it across the N ranks once. Each grid step, the grid federate writes the changed loads and generator setpoints to rank 0's standard input; rank 0 broadcasts them, every rank updates its part of the grid, the power flow is solved in parallel (PETSc with the MUMPS direct solver), and the bus voltages are gathered back to rank 0 and returned.

Table 5. pf_server protocol (one exchange per grid step)

Direction	Line	Meaning
to rank 0	LOAD <bus> <P MW> <Q Mvar>	New load on a bus
to rank 0	VSET <bus> <gen id> <V pu>	New generator voltage setpoint
to rank 0	SOLVE / QUIT	Apply the batch and solve/stop the server
from rank 0	@@RANK <rank> <host> <buses> <branches>	Once per rank at start-up: the node of each rank and the buses and branches it owns
from rank 0	@@READY nbus= nranks= t_setup=	Network read and partitioned, ready for the first step
from rank 0	@@RESULT ok= nbus= t_apply= t_solve=	Solve finished: converged or not, time to apply the changes, and time to solve
from rank 0	@@V <bus> <Vm pu> <Va deg> ... @@END	Voltage magnitude and angle of every bus, in ascending bus order, then the end of the reply

The grid federate writes the commands to the standard input of rank 0. Rank 0 answers on its standard output, which also carries messages from GridPACK, PETSc, and Open MPI, such as the Newton–Raphson iteration lines. Every reply from the server, therefore, starts with the marker @@. The grid federate reads only the lines that begin with @@ as data, treating all other lines as log text. The iteration lines that come before each @@RESULT are shown on the Inside page as the convergence of the power flow.

3.6 GridPACK dynamic simulation on MPI: dsf_server

`dsf_server` is the dynamic counterpart of `pf_server`, built on GridPACK's full-Y dynamic simulation module. It is started once per experiment with `mpirun -np N`. GridPACK solves the initial power flow, initializes every machine from its PSS/E dynamic data (.dyr), and then integrates the machine equations with a predictor-corrector method in steps of about 5 ms (seven steps per PMU frame). Faults and trips are scheduled in its input file; setpoint changes are sent while it runs.

Table 6. `dsf_server` protocol (one exchange per PMU frame)

Direction	Line	Meaning
to rank 0	DVREF <bus> <gen id> <dV pu>	move the exciter voltage reference to its start value + dV (commands, AVR faults)
to rank 0	STEP <t> / QUIT	integrate up to time t and report/stop
from rank 0	@@READY nbus= ngen= nranks= t_setup= dt=	machines initialised; integration step
from rank 0	@@RESULT t= nbus= ngen= t_apply= t_solve=	time reached, with timings
from rank 0	@@V <bus> <Vm> <Va> <f Hz>	voltage and frequency of every bus
from rank 0	@@G <bus> <id> <speed> <angle> <P> <Q> <on> ... @@END	state of every machine

Table 7. Dynamic models and events

Item	Supported
Machines	GENROU, GENSAL, GENCLS (and GridPACK's other models in a .dyr file)
Exciters/governors	IEEET1, SEXS, ESST1A, EXDC1 (reference changeable at run time) / TGOV1, GGOV1, HYGOV, WSIEG1 ...
Machine data	IEEE 39: published data (GENROU, IEEET1, TGOV1). Any grid: its own dynamics.dyr + dynamics.json (machine bases) in the grid directory. Otherwise, typical data: GENROU, SEXS exciter, TGOV1 governor on a machine base of 1.25 × output
Events	three-phase bus fault with clearing time, line trip, generator trip, and AVR setpoint fault
Loads	constant impedance

GridPACK 3.5 is built with a small patch (`gridpack/patches/dynamic-simulation.patch`): exciter references can be read and changed at run time, the network can be read directly so that `dsf_server` gathers voltages, frequencies and machine states on every rank (GridPACK's own observation functions lose buses and can hang on more than one rank), and a line trip no longer crashes on ranks that do not hold the line.

3.7 What uses MPI

Part	MPI?	Why
GridPACK	Yes	The power flow and network solution for each integration step are large systems of equations; GridPACK is built to split them across MPI ranks.
HELICS	No (ZeroMQ over TCP)	HELICS only synchronizes clocks and passes messages; ZeroMQ works on any network. HELICS also has an MPI core for supercomputers.
NS-3	No	The PMU network (up to thousands of PMUs) runs in seconds on one node. NS-3's MPI mode is useful for far larger networks.
Control center	No	Each estimation is small (23 ms for 10 000 buses with sparse matrices).

4. Installation

There are two ways to install the testbed, and both run the **same build scripts** (`install/steps/`), so they contain the same software in the same places: directly on Ubuntu 24.04 (native) or as a Docker container. GridPACK, HELICS, and NS-3 are compiled from source; the first installation takes about an hour.

4.1 Requirements

Item	Minimum	Recommended (single server)
Operating system	Ubuntu 24.04 x86-64	Ubuntu 24.04 LTS server
CPU	4 cores	16 cores
Memory	8 GB	32 GB
Disk	15 GB free	500 GB SSD (results accumulate)
Network	Internet access during the installation	VPN or SSH access for remote users

4.2 Layout on disk

Path	Contents	Size
/opt/cps/ga, gridpack, helics, ns-3	Global Arrays, GridPACK (+ pf_server, dsf_server), HELICS, NS-3 (+ network federate)	0.44 GB
/opt/cps/venv	Python environment (federates, API)	0.44 GB
/opt/cps/node-red	Node-RED user directory: flows, settings, Dashboard 2	0.14 GB
/opt/cps/testbed	Testbed code	2 MB
Ubuntu packages	PETSc, OpenMPI, Boost, compilers, Node.js, ...	1.6 GB
/srv/cps	Data: runs/, grids/, legacy/, cluster.json, hostfile	grows with use (Section 9)
/etc/cps/cps.env	Settings read by the services	—

4.3 Native installation on Ubuntu 24.04

```
git clone https://github.com/mazharulmd/cps-testbed-cluster.git
cd cps-testbed-cluster
sudo ./install/install.sh
```

The installer builds the software, creates the system user `cps`, writes `/etc/cps/cps.env`, installs the command `cps-run`, and starts two systemd services. At the end, it prints the addresses of the dashboard and the API.

Table 8. Installation steps (install/steps)

Step	What it does	Skipped on re-run?
00-packages	Ubuntu packages: compilers, OpenMPI, PETSc, ParMETIS, Boost, ZeroMQ, Python, Node.js, OpenSSH	no (apt is idempotent)
10-ga	Global Arrays 5.9 (GridPACK's data layer)	yes
20-gridpack	GridPACK 3.5 from GitHub with the testbed patches	Yes, unless the patches changed
30-helics	HELICS 3.6.1 C++ library and helics_broker	yes
40-ns3	NS-3.48 with the modules the network federate needs	yes
50-python	Python environment from requirements.txt	updated
60-nodered	Node-RED 4.1.11 and Dashboard 2	updated
70-testbed	Code, NS-3 federate, pf_server, dsf_server, library paths, Node-RED flows, data directories	always rebuilt

4.4 Settings

Table 9. `/etc/cps/cps.env`

Variable	Meaning	Default
CPS_ROLE	head (dashboard, API, brokers) or node (compute node)	head
CPS_NODES	MPI hosts and slots, e.g. node1:16, node2:16; empty on a single server	empty
CPS_PLACE_NS3, CPS_PLACE_CC, CPS_PLACE_GRID	Nodes that run the NS-3, control-center, and grid federates	the head
CPS_MPI_NP	Default GridPACK MPI ranks per experiment	1
CPS_MAX_JOBS	Experiments that may run at the same time	1
CPS_WEB_USER, CPS_WEB_PASSWORD	Login of the experiment API (and of the dashboard's calls to it)	admin/none
NODERED_ADMIN_USER, NODERED_ADMIN_HASH	Login of Node-RED (bcrypt hash)	admin/none
CPS_SHARED	Shared data directory	/srv/cps
OMPI_MCA_btl_tcp_if_include	Network interface for MPI between nodes	all

4.5 Logins

Without a password, anyone who can reach ports 1880 and 8080 can run experiments. Set one before giving others access:

```
sudo cps-passwd          # asks for the password twice; --off removes it
```

The same password then protects the dashboard, the Node-RED editor and the experiment API (user `admin`). `cps-passwd` stores it in `/etc/cps/cps.env` (for Node-RED as a bcrypt hash) and restarts both services. Use at least 8 characters and no quotes, backslashes or spaces.

4.6 Services

Command	Purpose
<code>systemctl status cps-api cps-nodered</code>	State of the API (port 8080) and Node-RED (port 1880)
<code>journalctl -u cps-api -f</code>	API and experiment queue log
<code>journalctl -u cps-nodered -f</code>	Node-RED log
<code>sudo systemctl restart cps-api cps-nodered</code>	Apply the changed settings
<code>systemctl status cps-node</code>	Compute node service (cluster)
<code>sudo -u cps cps-run --help</code>	Run experiments from the command line

4.7 Docker

The image is built from `Docker/Dockerfile` with the same steps, or pulled from the GitHub container registry, where the *Container image* workflow publishes it.

```
docker build -f docker/Dockerfile -t cps-testbed-cluster .  
# or: docker pull ghcr.io/mazharulmd/cps-testbed-cluster:latest  
  
docker compose -f docker/compose.yml up -d          # single server, one container  
docker compose -f docker/compose.cluster.yml up -d  # head + 2 compute nodes on  
one machine
```

Compose file	Containers	Use
<code>docker/compose.yml</code>	testbed (head role)	Single server; GridPACK on the container's cores
<code>docker/compose.cluster.yml</code>	head, node1, node2 (2 MPI slots each)	Try the cluster on one machine; GridPACK on 4 ranks over node1 and node2, NS-3 on node1, control center on node2

5. Setting up a cluster

5.1 Recommended hardware

Machine	Role	Hardware
Head (1)	Node-RED, API, HELICS brokers, grid federate; NFS server for /srv/cps	8 cores, 16–32 GB RAM, 500 GB–1 TB SSD
Compute nodes (2–4)	GridPACK MPI ranks; NS-3 on one node, control center on another	16–32 cores, 32–64 GB RAM, 100 GB SSD each
Network	Connects the machines	1 Gbit/s minimum, 10 Gbit/s better for MPI

5.2 Steps

1. Install every machine: `sudo ./install/install.sh` on the head, `sudo ./install/install.sh --role node` on each compute node.
2. Share `/srv/cps` from the head over NFS and mount it at `/srv/cps` on every node, writable for user `cps` (same UID on all machines).
3. Make the host names resolvable (DNS or `/etc/hosts`) and allow TCP between the machines (ssh, HELICS ports 23500 and up, MPI).
4. On the head set `CPS_NODES`, `CPS_PLACE_NS3`, `CPS_PLACE_CC`, `CPS_MPI_NP` and `CPS_MAX_JOBS` in `/etc/cps/cps.env` and run `sudo systemctl restart cps-api`.
5. The head creates the cluster SSH key in `/srv/cps/.ssh`; the `cps-node` service on each node installs it. `journalctl -u cps-api` shows "node1 reachable".
6. Check the *Cluster* panel on the Experiments page (Figure below). The *Cluster* page (Section 6.4) changes the nodes, their MPI slots, and the placement from the dashboard instead of `cps.env`.

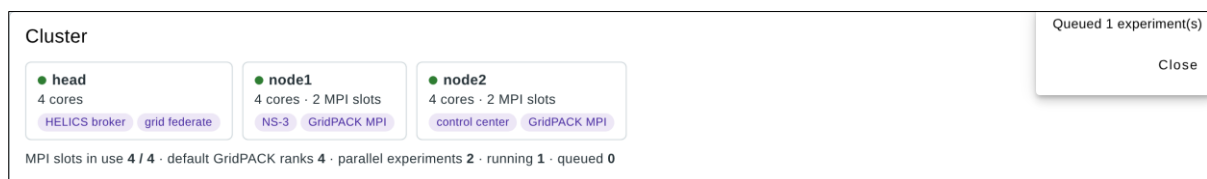


Figure 3. Cluster panel: the head runs the broker and the grid federate, node1 NS-3 and GridPACK ranks, node2 the control center and GridPACK ranks.

5.3 Scheduling

Up to `CPS_MAX_JOBS` experiments run at once, each with its own HELICS port. An experiment starts only when its GridPACK ranks fit in the free MPI slots; the others wait in the queue ("waiting for N MPI slots"). MPI uses busy-wait, so running more ranks than slots slowed two parallel runs by about 10× in a test (from 13 s to 150 s each).

6. Using the dashboard

6.1 Opening the dashboard

Address	What
http://<server>:1880/dashboard/experiments	Dashboard (Experiments, Live, Results, Cluster, IEEE 14 console)
http://<server>:1880	Node-RED editor (flows)
http://<server>:8080/docs	Experiment API and its interactive documentation

From outside the institution, use the VPN or an SSH tunnel, e.g., `ssh -L 1880:localhost:1880 -L 8080:localhost:8080 user@server`, then open `http://localhost:1880/dashboard/experiments`.

6.2 Experiments page

The page has five panels: *Cluster* (Figure in Section 5.2), *New experiment* (Section 6.3), *Upload files*, *Grid models*, and *Experiment queue*.

Upload

1. Grid model (optional): a MATPOWER case file `.m` or a testbed `topology.json`. The power flow is checked and the PMUs needed are computed; the grid then appears under *Grid models* with an id to use in scenario files. PSS/E `.raw`: convert it with MATPOWER's `psse2mpc` first.

2. Scenario: a YAML or JSON file describing one experiment, or several under `scenarios:`. Start from the [scenario template](#). Each experiment is queued and runs on the cluster; results appear on the [Results](#) page.

 Upload grid model (.m / .json)

 File Uploaded! Upload Another File

Queued 1 experiment: **live-demo** on IEEE 118-bus (4 MPI ranks).

Figure 4. Upload panel after a scenario file was uploaded: the experiment was validated and queued with 4 MPI ranks.

Grid models					
Grid id (use in scenario files)	Buses	Branches	Generators	PMUs (min)	Load MW
ieee14 built-in	14	20	5	4	259
ieee30 built-in	30	41	6	10	189
ieee39 built-in	39	46	10	13	6254
ieee57 built-in	57	80	7	17	1251
ieee118 built-in	118	186	54	32	4242
ieee300 built-in	300	411	69	87	23526
case_activsg2000-78531 case_ACTIVSg2000.m · 2026-10-01 18:15:29	2000	3206	392	512	67109

Figure 5. Grid models: the built-in IEEE grids and an uploaded 2000-bus grid with its ID, size, and the PMUs needed.

Experiment queue						
Experiment	Grid	Attack	MPI ranks	Status	Time	Log
live-demo live.yaml · 01/10/2026, 18:17:27	IEEE 118-bus	fdi-simple	4	running	3 s	[RUN] 20261001_181728_live-demo: IEEE 118-bus, 32 PMUs, attack=fdi-simple, GridPACK MPI ranks=4, HELICS port 23600
texas2000-fault doc_batch.yaml · 01/10/2026, 18:15:33	case_ACTIVSg2000	none	4	done	26 s	20261001_181612_texas2000-fault
ieee118-delay doc_batch.yaml · 01/10/2026, 18:15:33	IEEE 118-bus	delay	4	done	13 s	20261001_181559_ieee118-delay
ieee118-fdi-stealthy doc_batch.yaml · 01/10/2026, 18:15:33	IEEE 118-bus	fdi-stealthy	4	done	13 s	20261001_181638_ieee118-fdi-stealthy
ieee118-fdi-simple doc_batch.yaml · 01/10/2026, 18:15:33	IEEE 118-bus	fdi-simple	4	done	26 s	20261001_181534_ieee118-fdi-simple

Figure 6. Experiment queue: status, MPI ranks, run time, and the run directory of each experiment.

6.3 Running an experiment from the form

New experiment runs one experiment without writing a file. The buttons at the top fill the form with a ready-made scenario; every field can then be changed before pressing *Run experiment*. The form offers only what the chosen grid has: its generator buses for an AVR fault or a generator trip, its buses for a fault or a load step, its lines for a line trip.

New experiment

Ready-made:
Attack detected (IEEE 118)
Stealthy attack (IEEE 118)
Delay attack (IEEE 118)
5% packet loss (IEEE 118)
Fault (dynamic, IEEE 39)
Generator trip (dynamic, IEEE 39)

AVR fault + control (dynamic, IEEE 39)

Experiment

Name

Grid

Simulation

Duration (s)

GridPACK MPI ranks

4 MPI slots on 2 nodes. Small grids run fastest on 1 rank; more ranks help large grids.

Grid event

Event

Bus

Cleared after (s)

At time (s)

Cyber attack

Attack

Network and control center

Link latency (ms)

Packet loss (%)

PDC wait (ms)

PMU placement

☒ bad data detection
☒ voltage control

Run experiment
Download as scenario file

Figure 7. The New experiment panel after choosing the ready-made "Fault (dynamic, IEEE 39)".

Ready-made scenario	What it shows
Attack detected (IEEE 118)	AVR fault at generator 49 hidden by a simple false data injection; bad data detection finds the PMU, and the control acts
Stealthy attack (IEEE 118)	All channels of the target changed consistently; not detected
Delay attack (IEEE 118)	PMU frames around bus 49 are delayed by 100 ms; they miss the PDC window
5% packet loss (IEEE 118)	Lossy links without an attack
Fault (dynamic, IEEE 39)	Three-phase fault at bus 16, cleared after 0.1 s
Generator trip (dynamic, IEEE 39)	Generator 32 (650 MW) trips; frequency falls and recovers
AVR fault + control (dynamic, IEEE 39)	Generator 30 setpoint jumps to 1.12 pu; the control center corrects it

GridPACK MPI ranks lists the choices the cluster allows, with the default of the Cluster page marked. *Download as scenario file* saves the form as a YAML file that can be edited and uploaded later, for example, to run it again or to build a file with several experiments. After *running the experiment*, the answer appears under the buttons (or the reason the experiment was refused), with a link to the Live page.

6.4 Cluster page

The *Cluster* page shows the nodes and lets you change the cluster without editing files. *Save* applies the settings at once to experiments started afterward: MPI slots, the queue, and the number of experiments follow immediately, and nodes that do not respond over SSH are named. The settings are kept in `/srv/cps/cluster_settings.json` and take precedence over `/etc/cps/cps.env`; *Use the settings file* goes back to it.

Nodes

- head**
4 cores
HELICS broker, grid federate
- node1**
4 cores · 2 MPI slots
NS-3, GridPACK MPI
- node2**
4 cores · 2 MPI slots
control center, GridPACK MPI

MPI slots in use 4 / 4 · default GridPACK ranks 2 · parallel experiments 1 · running 1 · queued 1

Cluster settings

Single server: leave the node list empty. GridPACK then runs its MPI ranks on this server's 4 cores and every federate runs here.

Cluster: install each extra machine with `sudo ./install/install.sh --role node`, share `/srv/cps` from this server over NFS, then list the machines below with the cores GridPACK may use on each (MPI slots). The head sets up SSH to them when you save.

MPI and experiments

Default GridPACK MPI ranks per experiment

2

Experiments at the same time

1

MPI slots in all: 4. An experiment waits in the queue until its MPI ranks fit in the free slots.

Compute nodes (MPI hosts)

node1 2 remove

node2 2 remove

+ add node

Where the federates run

NS-3 network federate

node1

Control-center federate

node2

Grid federate (starts GridPACK)

head (head)

Save Use the settings file Now from: dashboard. Changes apply to experiments started afterwards.

Saved. MPI slots: 4.

Figure 8. Cluster page: the nodes with their roles, and the settings (default MPI ranks, experiments at the same time, compute nodes and their MPI slots, node of each federate).

Each experiment checks the nodes when it starts. Nodes that do not answer are left out of their MPI hostfile (their log names them); if a federate is placed on such a node, or the remaining slots are too few for the requested MPI ranks, the experiment stops at once with that message.

6.5 Scenario files

A scenario file describes one experiment or several under different scenarios (fields at the top are shared; each entry overrides them). Download the scenario template using the link in the Upload panel.

```
name: gen-fault-fdi
grid: ieee118          # ieee14 ... ieee300 or an uploaded grid id
mode: qss              # qss (power flow) or dynamic
duration: 10
mpi_np: 4              # GridPACK MPI ranks
pmu: {placement: minimum, rate: 30}
network: {latency_ms: 10, latency_spread: 0.5, jitter_ms: 2, loss: 0.0, pdc_wait_ms: 20}
event: {type: avr_fault, bus: 49, vset: 1.12, t: 4}
attack: {type: fdi-simple, target: 49, fake: 1.02, start: 4, end: 9}
control: {bdd: true, voltage_control: true, vmin: 0.94, vmax: 1.08}
```

Table 10. Scenario fields

Field	Meaning	Values (default)
name	Run name, used in the results folder	letters, digits, _ - (scenario)
grid	Grid to simulate	ieee14...ieee300 or uploaded id (ieee14)
mode	Quasi-steady state (power flow) or dynamic simulation	qss / dynamic (qss)
duration, grid_step	Simulated seconds; grid step (power flow interval, or one PMU frame in dynamic mode)	1–120 (10); 0.5 s or 1/rate
solver, mpi_np	GridPACK with N MPI ranks, or the built-in solver	gridpack / builtin; 1 to the MPI slots (Cluster page default)
pmu. placement, PMU. rate	Minimum or redundant placement; frames per second	minimum / redundant; 1–120 (30)
network.*	Link latency, spread, jitter, loss, PDC wait window	10 ms, 0.5, 2 ms, 0, 20 ms
event.*	AVR fault (setpoint), load step (percent, qss); bus fault (duration), line trip (bus, to), generator trip (dynamic); at time t	none / avr_fault / load_step / bus_fault / line_trip / gen_trip
attack.*	FDI simple or stealthy, delay or drop; target bus, fake voltage, window	none / fdi-simple / fdi-stealthy / delay/drop
control.*	Bad data detection, voltage control, and voltage limits	true, true, 0.94, 1.08

Every field is checked before anything runs. A wrong value gives a clear message, for example, "bus 50 is not a generator bus in IEEE 118-bus; generator buses: 1, 4, 6, ...".

6.6 Uploading a grid model

Upload a MATPOWER case file (.m, the format of MATPOWER and of most published test systems) or a testbed `topology.json`. The testbed removes isolated buses, checks that the power flow converges (starting from the voltages stored in the case), computes the minimum PMU placement, and lists the grid with an ID such as `case_activsg2000-78531`. PSS/E .raw files are converted with MATPOWER first: `mpc = psse2mpc('grid.raw'); savecase('grid.m', mpc)`.

6.7 Live page

While an experiment runs, the Live page follows it every second. The data comes from the observer federate, which listens to the co-simulation without slowing it.

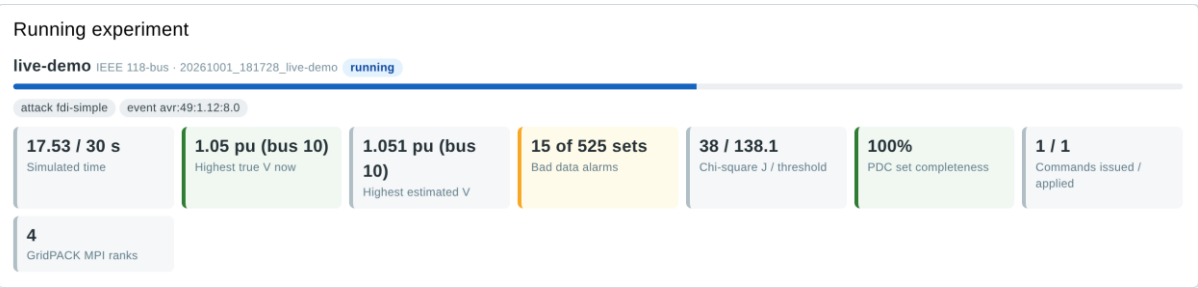


Figure 9. Live: progress, highest true and estimated voltage, alarms, chi-square value, PDC completeness, commands and MPI ranks, 12 s into a 30 s run.



Figure 10. Live charts during the run: the fault at 8 s, the falsified channel making the chi-square value jump, and the correction. Enter a bus number to follow.

Commands so far

Kind	Generator	Setpoint (pu)	Issued (s)	Delivered (s)	Applied (s)	Reason
event	49	1.12			8	event: AVR setpoint fault
command	49	1.015	8.067	8.107	8.5	V49=1.122

Figure 11. Commands so far: the AVR fault event, and the command issued at 8.067 s, delivered over the network at 8.107 s, and applied by GridPACK at 8.5 s.

The panel *Voltage at every bus now* shows the whole grid at the last estimate (section 6.9 explains the chart).

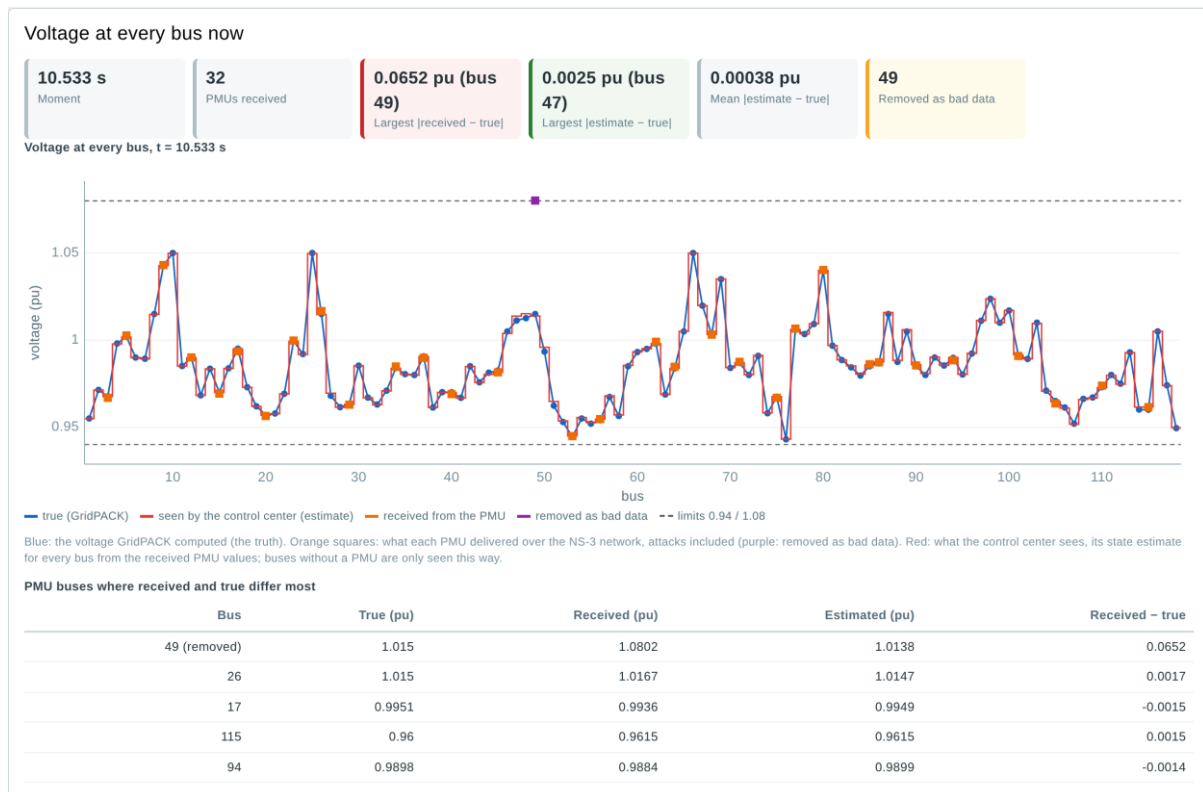


Figure 12. Voltage at every bus now, 10.5 s into an IEEE 118 run during a false data injection on bus 49: PMU 49 (purple) reports 1.080 pu for a true 1.015 pu; it is removed as bad data, and the estimate (red) follows the truth (blue).

6.8 Inside page: what the engines compute

The Inside page opens the three engines while an experiment runs, showing what each one is computing and updating every second. The figures below were taken 12 s into a 30 s IEEE 118 run on the virtual cluster: an AVR fault at generator 49 at 6 s, a simple false data injection on bus 49 from 8 to 18 s, 1% packet loss, and GridPACK on 2 MPI ranks.

Co-simulation (HELICS). A diagram of the federation: each federate with the node it runs on and the simulated time the HELICS broker has granted it, and each topic with the messages and bytes exchanged so far. HELICS keeps all federates on a single simulated clock; a federate is granted its next time only when the data it needs has arrived, so the granted times remain within one step of each other.

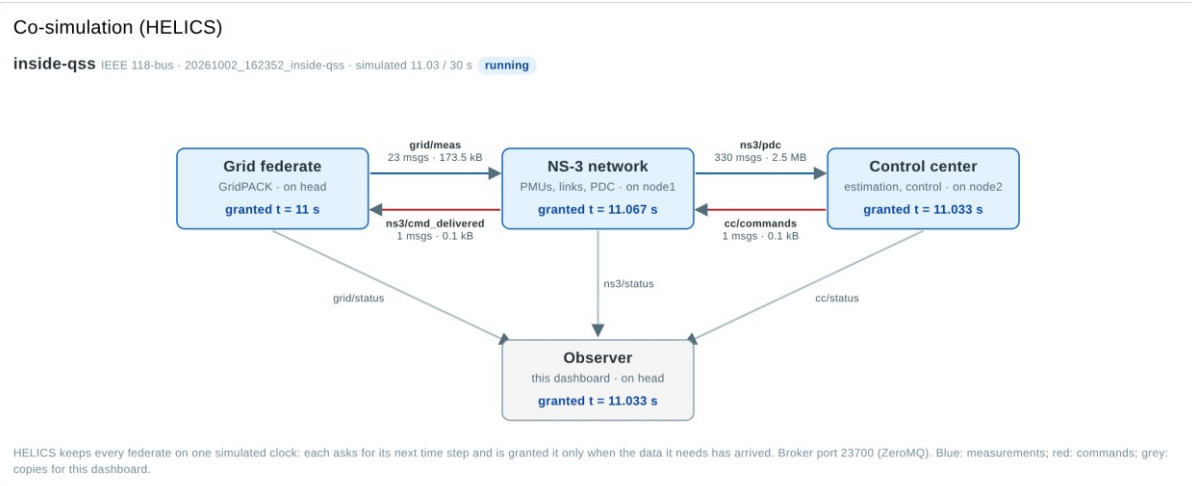


Figure 13. HELICS panel: grid federate on the head, NS-3 on node1, control center on node2, observer on the head, with their granted times; blue arrows carry measurements, red arrows commands, grey arrows copies for the dashboard.

GridPACK. The solver (pf_server for power flow, dsf_server for dynamic simulation), the number of MPI ranks, and how GridPACK partitioned the grid over them (node, buses, and branches of each rank), the times of the last step, and a chart of the time per step. In power flow mode, the chart in the middle is the Newton-Raphson convergence of the last power flow: the largest power mismatch after each iteration on a logarithmic scale, against the tolerance of 10^{-6} pu. In dynamic mode, the panel shows the integration steps of the frame instead (for example, 7 steps of 4.76 ms).

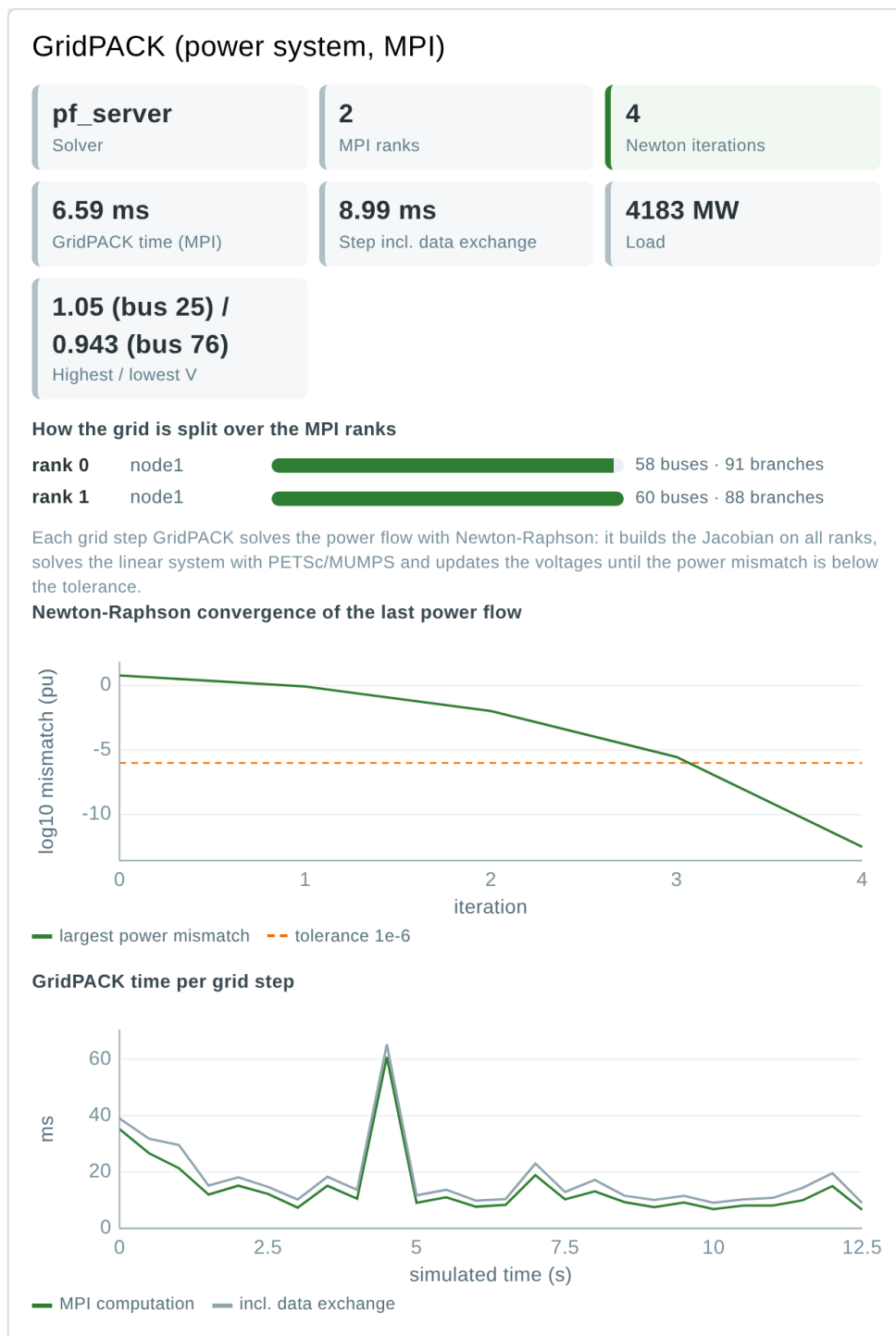


Figure 14. GridPACK panel in power flow mode: 2 ranks on node1 with 58 and 60 buses; Newton-Raphson requires 4 iterations, and the mismatch decreases from 5.9 to 3×10^{-13} pu.

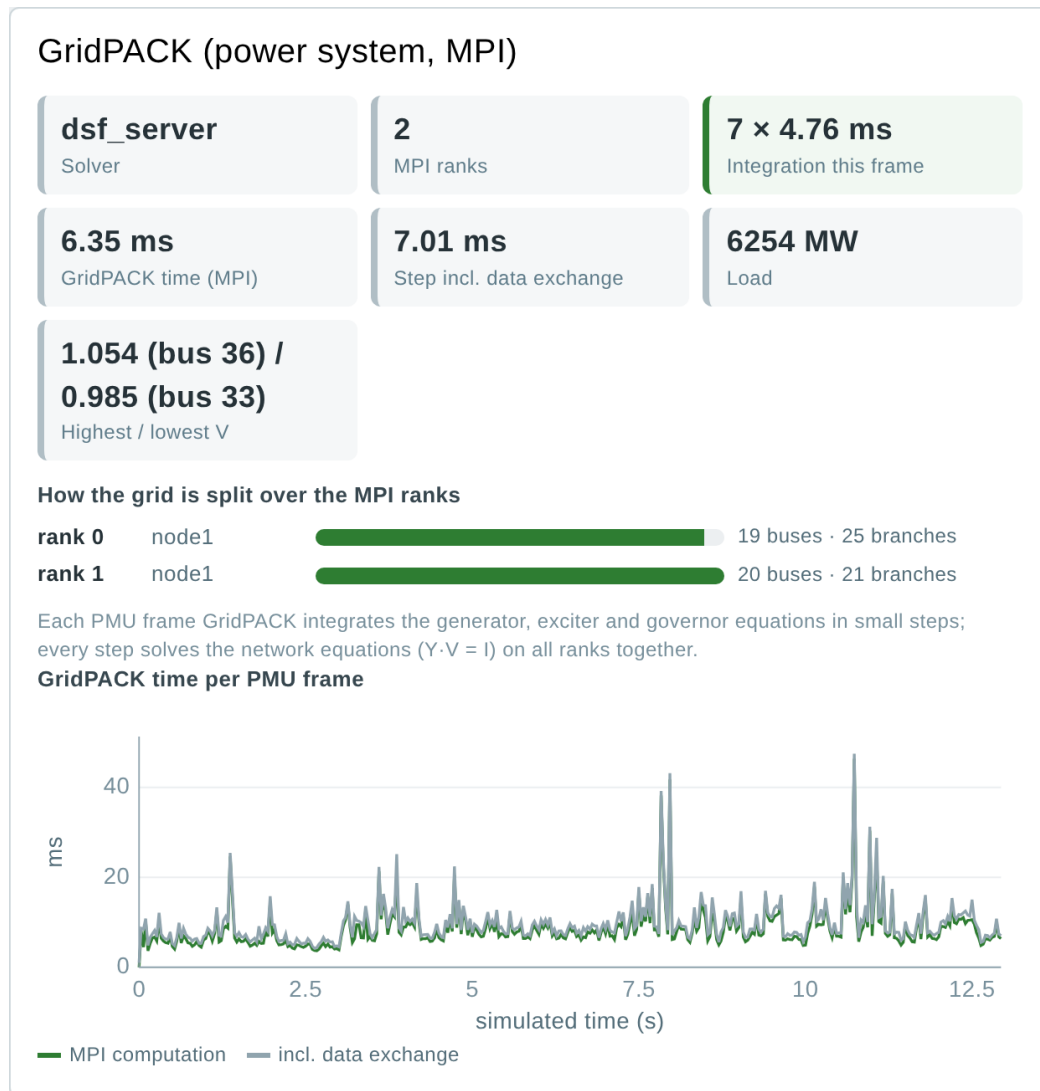


Figure 15. GridPACK panel in dynamic mode (IEEE 39, fault at bus 16 at 3 s): 7 integration steps of 4.76 ms per PMU frame, 6-10 ms of MPI time per frame.

NS-3. Frames sent by all PMUs and the share that reached the PDC in time; late, dropped, and attacked frames; PDC sets released and how many were complete; the time the PDC waited; commands sent and delivered; the attacker; and the number of events the network simulator has processed. Each PMU is a tile colored by what happened to its frames in the last 0.5 s (hover over it for its counters), and a chart shows the latency of the last frame for each PMU.

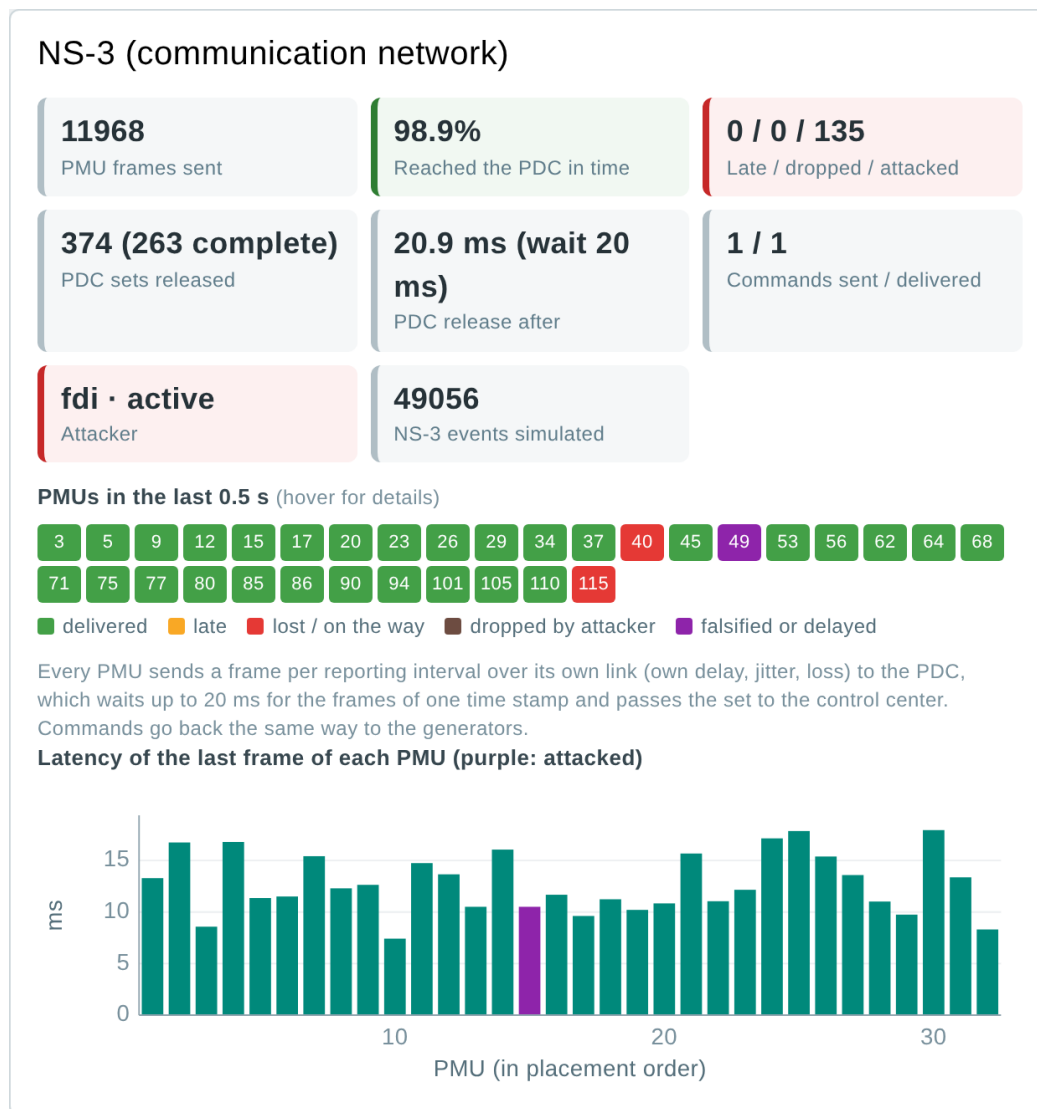
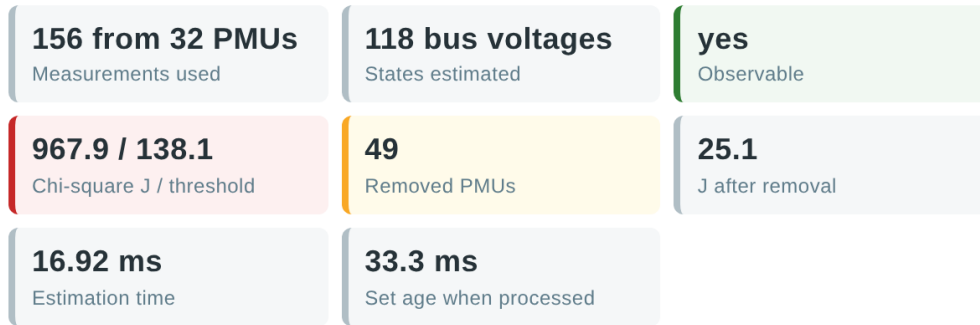


Figure 16. NS-3 panel: 98.9% of the frames arrive in time (1% loss: PMUs with a lost frame in the last 0.5 s are red); PMU 49 is attacked (purple); the PDC waits up to 20 ms for each set.

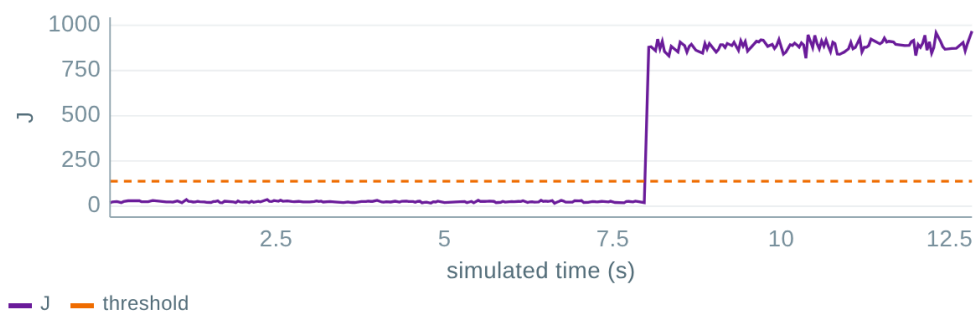
Control center. For the last measurement set: the measurements and PMUs used, whether the grid was observable, the chi-square value J against its threshold, the PMUs removed as bad data, J after the removal, the estimation time, and the age of the set when it was processed. With the minimum PMU placement, every PMU is needed, so a set that loses a frame leaves part of the grid unobserved; such a set cannot be tested for bad data and is skipped in the chart.

Control center (state estimation)



For every PMU set the control center solves the weighted least-squares state estimate $z = H \cdot V$, tests the residuals with chi-square (J above the threshold means bad data), removes the PMU with the largest normalized residual and re-estimates, then checks the voltages and sends setpoint commands when one is out of limits. A set that misses PMUs can leave part of the grid unobserved (minimum placement has no spare PMU); such a set cannot be tested for bad data and the chart skips it.

Bad data test per complete set



Estimation time per set

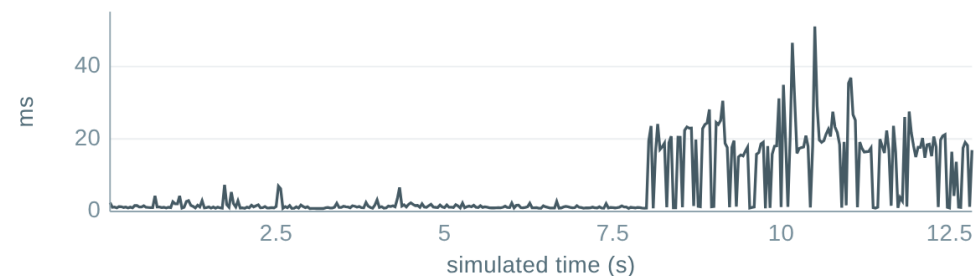


Figure 17. Control center panel: during the attack, J is about 900 against a threshold of 138; PMU 49 is removed, and J falls to 25; with the residual test and the second estimate, a set takes 10-40 ms instead of about 1 ms.

Log. Everything that happens, in plain words, with the simulated time and the engine. *Key events* hold grid events, commands from issue to application, the attacker starting and stopping, bad data alarms, and the PMUs removed. Every step summarizes each power flow or integration and each half-second of network traffic.

What is happening now

Key events

8.033 s	Control center	chi-square $J = 879.6 > \text{threshold } 138.1$: bad data; removed PMU(s) 49, J after removal 18.6
8 s	NS-3	the attacker starts (fdi)
6.5 s	GridPACK	applies the command for generator 49: voltage setpoint 1.015 pu (issued 6.067 s, delivered 6.107 s)
6.107 s	NS-3	command for generator 49 delivered after 40 ms
6.067 s	Control center	V49=1.120 out of limits: command generator 49 to 1.015 pu, sent over the network
6 s	GridPACK	grid event: AVR setpoint fault at bus 49, setpoint 1.12 pu
0 s	GridPACK	pf_server runs on 2 MPI rank(s): node1 ranks 0, 1; the 118-bus grid is split among them

Every step (latest first)

12.5 s	GridPACK	power flow for 4183.1 MW of load on 2 rank(s) in 6.59 ms: Newton converged in 4 iteration(s), mismatch $5.9\text{e}+00 \rightarrow 8.2\text{e}-01 \rightarrow 1.0\text{e}-02 \rightarrow 2.9\text{e}-06 \rightarrow 3.2\text{e}-13$; highest V 1.05 pu at bus 25
12.5 s	NS-3	480 PMU frames sent in the last 0.5 s: 478 reached the PDC in time, 0 late, 2 lost or on the way, 15 attacked; lowest delivery PMU 40 93%, PMU 115 93%; PDC released 374 sets so far, 263 complete
12 s	GridPACK	power flow for 4193.5 MW of load on 2 rank(s) in 15.03 ms: Newton converged in 4 iteration(s), mismatch $5.9\text{e}+00 \rightarrow 8.2\text{e}-01 \rightarrow 1.0\text{e}-02 \rightarrow 3.0\text{e}-06 \rightarrow 3.3\text{e}-13$; highest V 1.05 pu at bus 10
11.5 s	GridPACK	power flow for 4203.6 MW of load on 2 rank(s) in 9.95 ms: Newton converged in 4 iteration(s), mismatch $5.9\text{e}+00 \rightarrow 8.2\text{e}-01 \rightarrow 1.0\text{e}-02 \rightarrow 3.0\text{e}-06 \rightarrow 3.6\text{e}-13$; highest V 1.05 pu at bus 10
11.5 s	NS-3	480 PMU frames sent in the last 0.5 s: 475 reached the PDC in time, 0 late, 5 lost or on the way, 15 attacked; lowest delivery PMU 5 93%, PMU 20 93%, PMU 56 93%; PDC released 344 sets so far, 240 complete
11 s	GridPACK	power flow for 4216.9 MW of load on 2 rank(s) in 8.07 ms: Newton converged in 4 iteration(s), mismatch $5.9\text{e}+00 \rightarrow 8.2\text{e}-01 \rightarrow 1.0\text{e}-02 \rightarrow 3.1\text{e}-06 \rightarrow 3.6\text{e}-13$; highest V 1.05 pu at bus 10

Figure 18. Log of the run: the AVR fault at 6 s, the command for generator 49 issued at 6.067 s, delivered after 40 ms, and applied at 6.5 s, the attacker at 8 s, and the alarm one frame later.

Table 11. Where does the Inside data come from

Engine	Source
HELICS	The observer federate asks the broker for the granted time of every federate (global_time query) and counts the messages and bytes of each topic it receives
GridPACK	pf_server and dsf_server print each rank's node and partition at start-up (@@RANK), PETSc's Newton iteration lines and the integration steps; the grid federate reads them and publishes them with its step on grid/status
NS-3	The NS-3 federate publishes its counters on ns3/status twice a second

Control center	The control-center federate adds the size, observability, test results and time of each estimation to cc/status
----------------	---

6.9 Results page

Runs								
Run	Grid	Attack	MPI ranks	GridPACK ms/step	Violation (s)	Detection	Delivery	Wall (s)
live-demo 20261001 181728	IEEE 118-bus	fdi-simple	4	279.88	0.5	4.2%	100.0%	34.1
ieee118-fdi-stealthy 20261001 181638	IEEE 118-bus	fdi-stealthy	4	241.1	0.5	0%	100.0%	11.9
texas2000-fault 20261001 181612	case_ACTIVSg2000	none	4	532.58	0.5	–	100.0%	21.3
ieee118-delay 20261001 181559	IEEE 118-bus	delay	4	210.35	5.5	0%	96.9%	11.6
ieee118-fdi-simple 20261001 181534	IEEE 118-bus	fdi-simple	4	249.67	0.5	10%	100.0%	24
cluster-fdi 20261001 175428	IEEE 118-bus	fdi-simple	4	222.19	0.5	10%	100.0%	13.3

Figure 19. Runs: one row per finished experiment; click a row to open it.

Run summary

ieee118-fdi-simple 20261001_181534_ieee118-fdi-simple

IEEE 118-bus

attack fdi-simple

loss 0

latency 10 ms

BDD on

control on

event avr:49:1.12:4.0

minimum PMUs

100%

Frames delivered

17.8 ms

Latency p95

100%

Complete PDC sets

0.00025 pu

SE error (clean)

0.5 s

True violation time

1

Control commands

0%

False alarms

10%

Attack detected

0.033 s

Time to detect

49

Compromised PMUs

1.026 / 1.026

Target V true / seen

4

GridPACK MPI ranks

249.7 ms

GridPACK solve / step

24 s

Wall time

Nodes: broken: head - grid: head - ns3: node1 - cc: node2 - MPI hosts: node1, node2

Figure 20. Run summary of the IEEE 118 simple FDI run, with color-coded figures (green good, amber borderline, red problem).

Table 12. Summary figures

Figure	Meaning
Frames delivered, Latency p95	Share of PMU frames that reached the PDC in time; 95th percentile of their delay
Complete PDC sets	Share of measurement sets with every PMU present
SE error (clean)	Mean difference between estimated and true voltages outside the attack window
True violation time	How long was any real bus voltage outside the limits
Control commands	Setpoint commands applied to the grid
False alarms	Alarms of the bad data test outside the attack window
Attack detected, Time to detect	Share of attacked sets with an alarm; delay to the first alarm
Compromised PMUs, Target V true / seen	PMUs, the attacker changed; true and estimated voltage of the target bus
GridPACK MPI ranks, solve / step, Wall time	Ranks used, mean MPI solve time per grid step, total run time

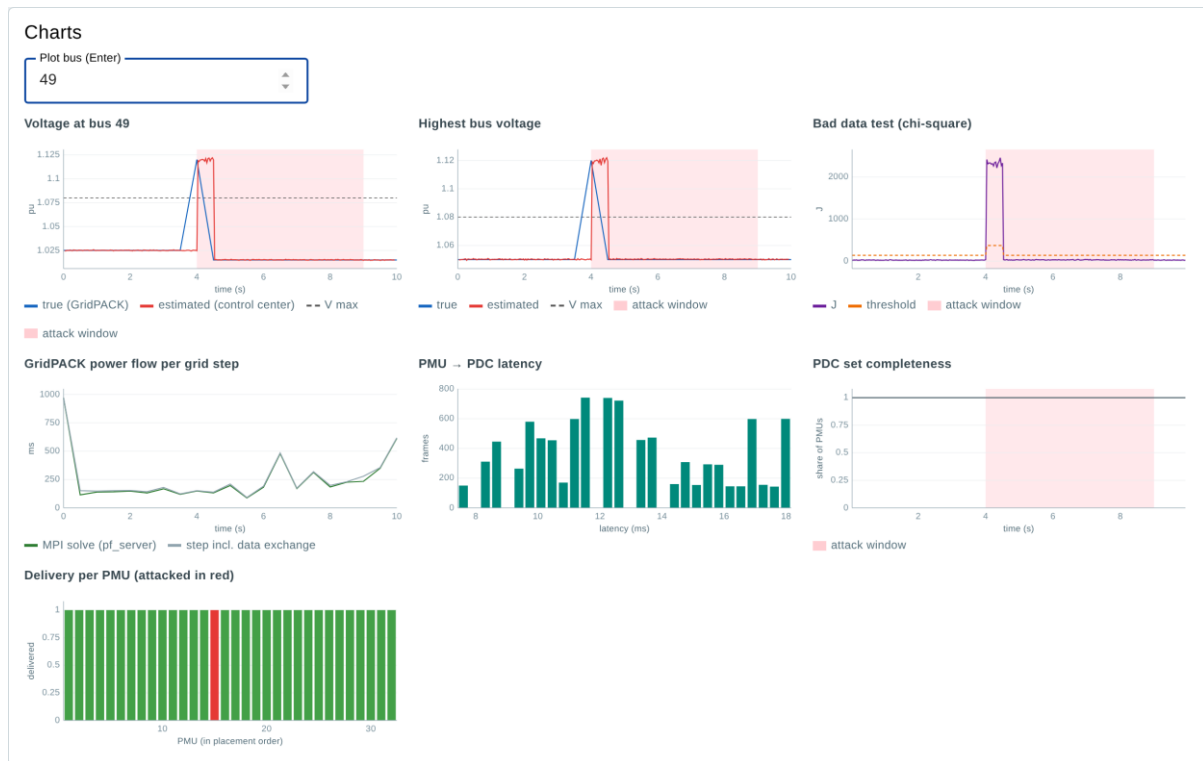


Figure 21. Result charts: voltage at bus 49, highest voltage, chi-square test, GridPACK solve time per grid step, PMU-to-PDC latency, PDC completeness and delivery per PMU (attacked PMU in red).

Commands and events					
Applied (s)	Generator	New setpoint (pu)	Issued (s)	Delivered (s)	Reason
4.0	49	1.12			event: AVR setpoint fault
4.5	49	1.015	4.066667	4.107	V49=1.119

Figure 22. Commands and events of the run.

Voltage at every bus

The panel Voltage at every bus compares, for one moment of the run, what the grid really does with what reaches the control center, as Fig. 8 of the technical report of the single-server testbed did for the IEEE 14 console:

- **Blue line (true):** the voltage GridPACK computed for every bus, for the grid state, the PMUs of that frame sampled.
- **Orange squares (received):** the bus voltage each PMU delivered over the NS-3 network, as it arrived, attacks included. Only buses with a PMU have one. A purple square was removed as bad data.
- **Red step line (seen):** the control center's state estimate for every bus, computed from the received PMU values. Buses without a PMU are only seen this way.

The slider and the arrows move through the run-one grid step by step (one PMU frame in dynamic mode); the buttons jump to the grid event, into the attack, and to the end. Figures above the chart give the largest difference between received and true and between estimated and true, and a table lists the PMU buses where received and true differ most.



Figure 23. IEEE 14, simple false data injection on bus 8 from 3 to 8 s, minimum placement, at 4 s. The attacker changes the current that the PMU at bus 7 measures towards bus 8. This is a critical measurement, so the chi-square test cannot see it ($J = 1.2$ against 13.3): the control center sees bus 8 at 1.040 pu while it is at 1.080 pu.

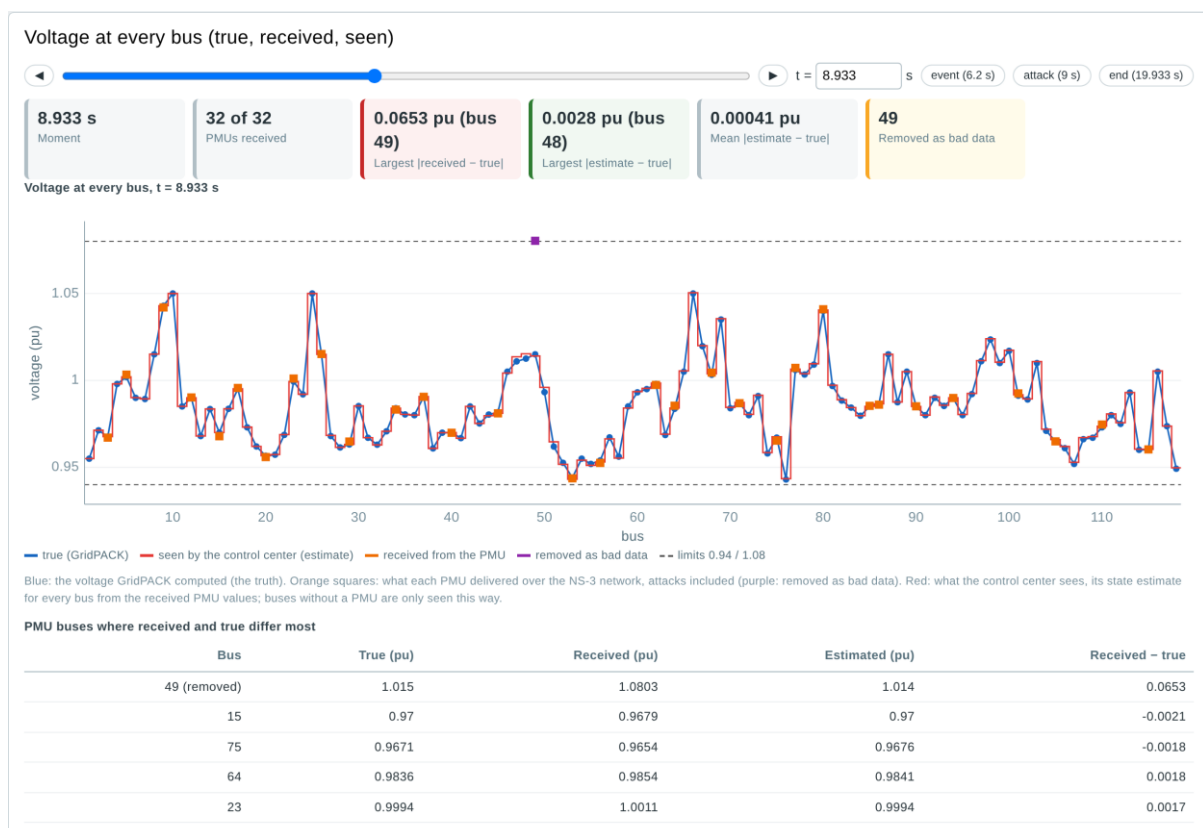


Figure 24. IEEE 118, simple false data injection on bus 49, at 8.9 s. PMU 49 delivers 1.080 pu for a true 1.015 pu; the test finds it and removes it, and the estimate stays within 0.003 pu of the truth at every bus.

6.10 Dynamic simulation runs

A scenario with mode: dynamic uses the same pages. Below, a file containing two IEEE 39 experiments (a 100 ms fault at bus 16 and the trip of generator 32, both at 3 s) was uploaded; both ran simultaneously. The Live page adds the system frequency, the lowest frequency so far, and the rotor angle spread, and two charts.

```
grid: ieee39
mode: dynamic
duration: 15
mpi_np: 1
scenarios:
- name: ieee39-dyn-fault
  event: {type: bus_fault, bus: 16, duration: 0.1, t: 3}
- name: ieee39-dyn-gentrip
  event: {type: gen_trip, bus: 32, t: 3}
```

Running experiment

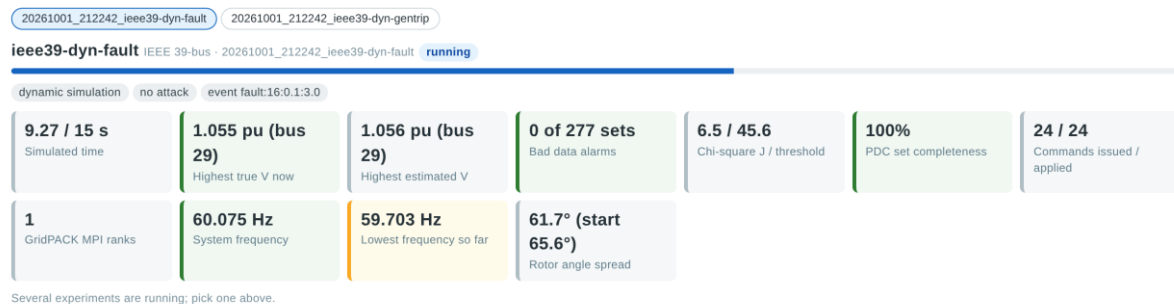


Figure 25. Live, dynamic run: frequency of the center of inertia, lowest frequency so far, and rotor angle spread next to the usual figures.

Live charts

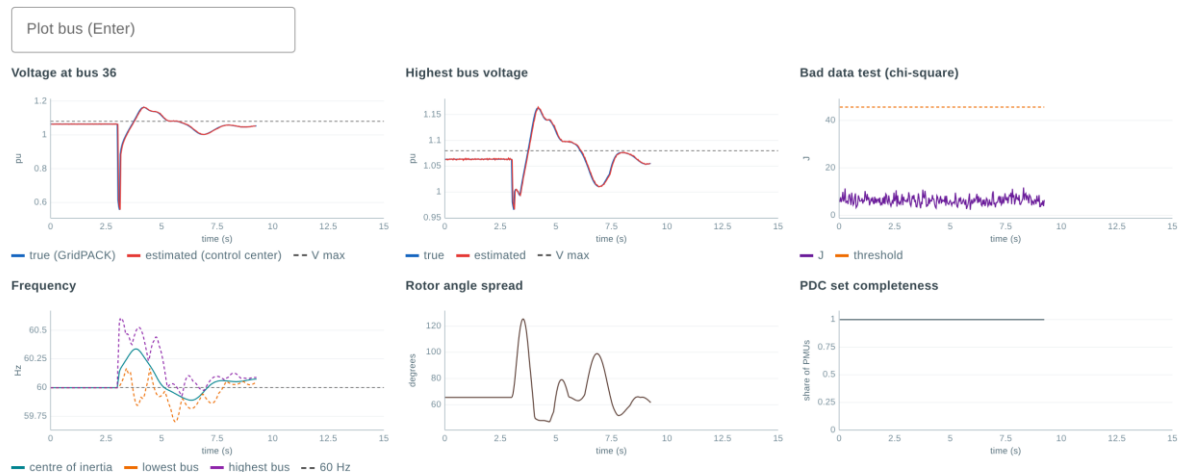


Figure 26. Live charts at 9.3 s: the fault at 3 s pulls bus 36 down to 0.56 pu; the exciters overshoot; frequency and rotor angles swing and settle.

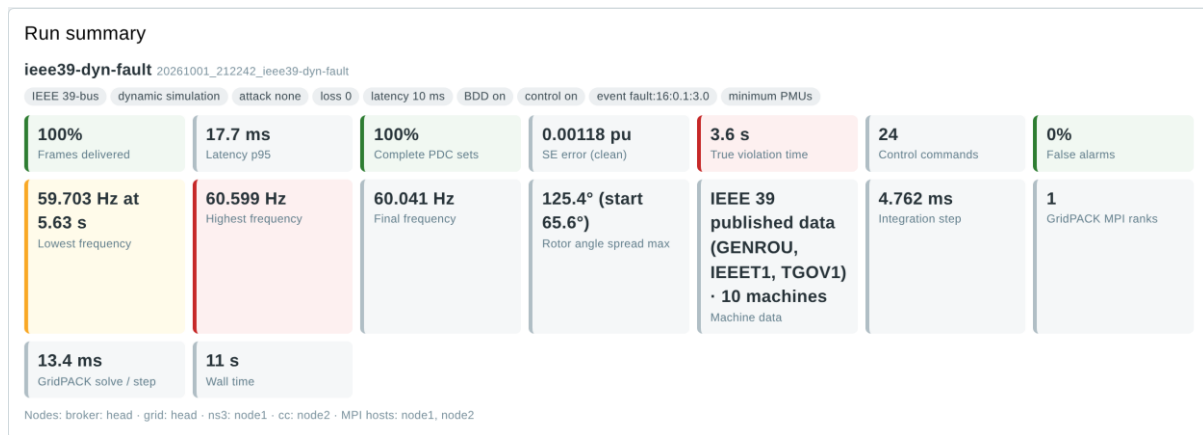


Figure 27. Results summary of the fault run: frequency range, final frequency, largest rotor angle spread, machine data, and integration step.



Figure 28. Results charts of the generator trip: frequency falls to 59.42 Hz and is arrested by the governors; GridPACK needs about 14 ms per frame on one rank.

6.11 IEEE 14 console (original prototype)

The original single-PMU IEEE 14 demonstration is kept on its own page. Set latency, loss, a bus to drop or the FDI switch and press *Run scenario*; the console updates in about 15 seconds.



Figure 29. Run Scenario panel of the IEEE 14 console.

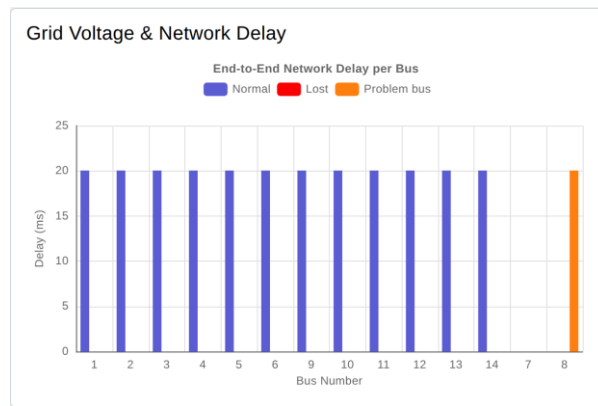


Figure 30. End-to-end network delay per bus. The voltage chart beside it loads its chart library from the internet and stays empty on servers without internet access.

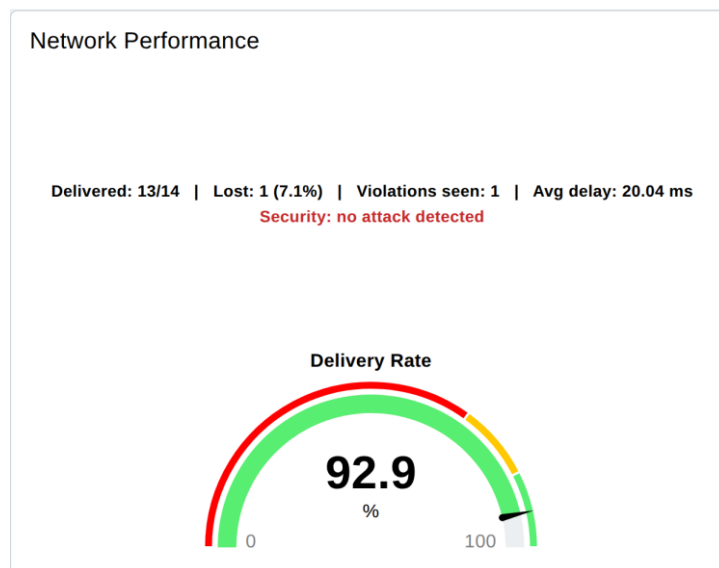


Figure 31. Network performance and delivery gauge.

Per-Bus Measurement Table

Bus	True V	Recv V	Delay ms	Status
1	1.0600	1.0600	20.03	OK
2	1.0450	1.0450	20.03	OK
3	1.0100	1.0100	20.03	OK
4	1.0177	1.0177	20.04	OK
5	1.0195	1.0195	20.04	OK
6	1.0700	1.0700	20.03	OK
7	1.0615	—	—	LOST
8	1.0900	1.0900	20.03	OK
9	1.0559	1.0559	20.04	OK
10	1.0510	1.0510	20.04	OK
11	1.0569	1.0569	20.04	OK
12	1.0552	1.0552	20.04	OK
13	1.0504	1.0504	20.04	OK
14	1.0355	1.0355	20.04	OK

Figure 32. Per-bus measurement table: true and received voltage, delay, and status.

6.12 Node-RED editor

Administrators can change the dashboard in the Node-RED editor without programming: the *CPS Cluster* tab holds the Experiments, Live, and Results pages, the CPS Testbed Console tab holds the IEEE 14 console.

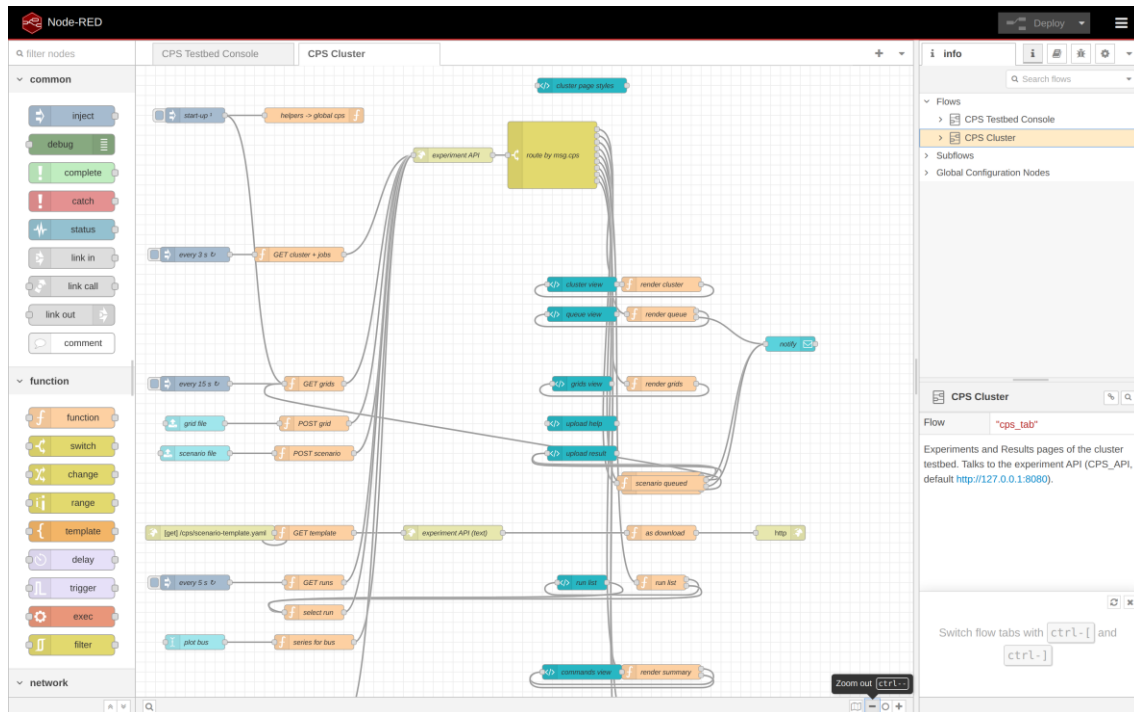


Figure 33. The CPS Cluster flow in the Node-RED editor.

6.13 API and command line

Table 13. Main API endpoints

Endpoint	Purpose
POST /api/scenarios/file	Upload a scenario file (multipart); queues its experiments
POST /api/grids/file	Upload a grid model (multipart)
GET /api/grids, GET /api/cluster, GET /api/jobs	Grids, cluster nodes and slots, queue
GET /api/runs, GET /api/runs/{id}, GET /api/runs/{id}/series	Finished runs, one run, chart data
GET /api/runs/{id}/profile?t=9	Voltage of every bus at one moment: true, received from the PMUs, estimated
GET /api/live	State of the running experiment
GET /api/scenario-template	The scenario template

```
curl -u admin:PASSWORD -F file=@scenario.yaml
http://server:8080/api/scenarios/file
sudo -u cps cps-run --case 118 --duration 10 --mpi-np 4 --event avr:49:1.12:4 \
--attack fdi-simple --target 49 --fake 1.02 --attack-start 4 --attack-end 9
sudo -u cps cps-run --case 39 --mode dynamic --duration 15 --event
fault:16:0.1:3
```

For reports, `tools/plot_profile.py` draws the voltage of every bus at one moment of a finished run as a PNG (without `--t`: one second into the attack, or just after the grid event):

```
sudo -u cps /opt/cps/venv/bin/python /opt/cps/testbed/tools/plot_profile.py \
/srv/cps/runs/20261003_031629_inside-dyn --t 3.07 --out fault39.png
```

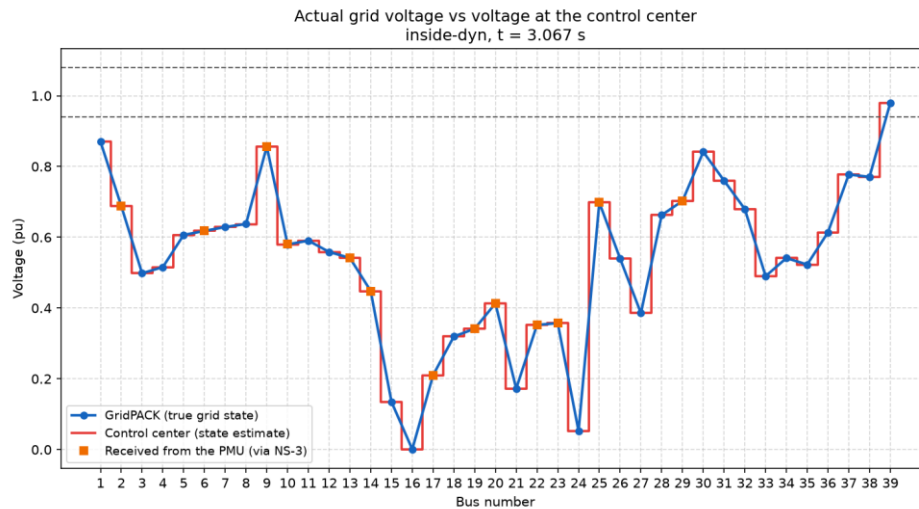


Figure 34. `plot_profile.py` on the dynamic IEEE 39 run, 67 ms after the three-phase fault at bus 16: bus 16 is at 0 pu, its neighbors between 0.1 and 0.6 pu; the PMU values and the estimate lie on the true curve.

7. Output files

Each experiment writes a directory `/srv/cps/runs/<date>_<time>_<name>/. The CSV files open in Excel, Python or MATLAB.`

File	Contents
summary.json	All summary figures (network, grid, control center, attack, GridPACK, cluster)
meta.json, run_config.json, ns3_config.json	Settings, PMU placement with channels, attack plan, placement of the federates
frames.csv	Every PMU frame: time stamp, PMU, send and arrival time, latency, status, attacked
cc_log.csv, cc_estimates.csv	Every measurement set: chi-square test, alarms, removed PMUs, violations; estimated voltage of every bus
cc_received.csv	Every measurement set: the bus voltage each PMU delivered, as received (attacks included); empty when its frame was missing
grid_truth.csv, grid_steps.csv	True voltage of every bus each step; solver, convergence, load, solve times, MPI ranks
commands_applied.csv	Events and commands applied to the grid, with issue, delivery and application times
live.json	Final state of the live view
grid_dynamics.csv, dynamics.json	Dynamic mode: frequency (centre of inertia, lowest and highest machine and bus), rotor angle spread, speed, angle and power of every machine each frame; machine data used and integration step
gridpack/	pf_server or dsf_server input (.raw, .dvr, input.xml) and log
*.log	Output of the broker and each federate

8. Validation and results

8.1 Test environments

Installation	Environment	Result
Native, install/install.sh	Fresh Ubuntu 24.04 with systemd, single server	Services running as user cps; experiments, CLI, legacy console, grid upload and update (53 s, results kept) work
Docker, compose.yml	One container	Experiment and legacy console work
Docker, compose.cluster.yml	Head + 2 nodes on one 4-core machine	Federates and MPI ranks on the nodes; parallel experiments; dashboard pages work

8.2 IEEE 118-bus on the cluster

IEEE 118-bus, 10 s, AVR fault at generator 49 (setpoint 1.12 pu at 4 s), attack from 4 to 9 s, GridPACK on 4 MPI ranks over node1 and node2, NS-3 on node1, control center on node2.

Attack	Violation	Detection	Complete sets	What happened
Simple FDI on bus 49	0.5 s	in 0.033 s	100%	Falsified PMU 49 found and removed; control acted within one grid step
Stealthy FDI on bus 49	0.5 s	none	100%	PMUs 45 and 49 changed consistently; not detected (seen 1.08 pu), but the violation was corrected
Delay 100 ms on PMUs 45, 49	5.5 s	—	49.7%	Frames missed the PDC window; the violation was seen late
5% packet loss (separate run)	0.5 s	—	18.1%	Estimation stayed usable

All federates (broker, grid, NS-3, control center, observer) finished with exit code 0 in every run; 9568 PMU frames were sent per run, p95 latency 17.8 ms, state estimation error 0.00025 pu, no false alarms. The results agree with the single-server testbed.

8.3 2000-bus Texas grid

The ACTIVSg2000 synthetic Texas grid (2000 buses, 3206 branches, 392 generator buses) was uploaded as a MATPOWER file; the testbed placed 512 PMUs. With an AVR fault at generator 1004, 76 288 PMU frames were delivered and the violation was corrected within one grid step.

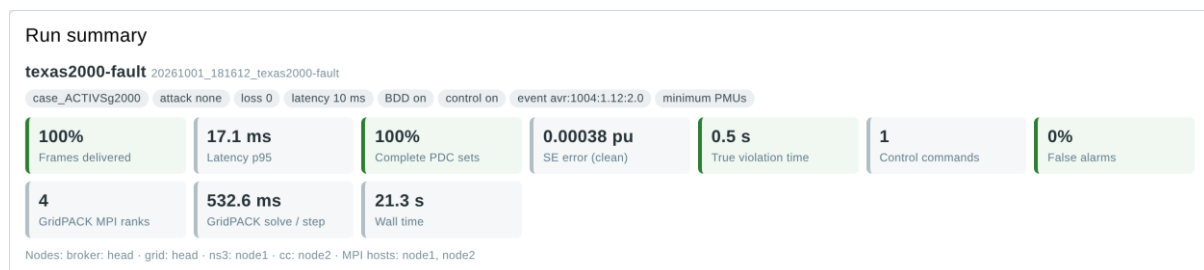


Figure 35. Summary of the 2000-bus run on the cluster.



Figure 36. Charts of the 2000-bus run.

8.4 GridPACK on MPI: accuracy and time

pf_server was compared with the testbed's own Newton-Raphson solver at four operating points with load and setpoint changes.

Grid	Max. difference	1 rank	2 ranks	4 ranks
IEEE 118, one container	5×10^{-10} pu	6–16 ms	15–27 ms	15–31 ms
IEEE 118, 4 ranks over 2 containers	5×10^{-10} pu	–	–	61–71 ms
ACTIVSg2000, one container	5×10^{-10} pu	104–120 ms	78–116 ms	55–97 ms
ACTIVSg10k, one container	6×10^{-5} pu	526–638 ms	452–624 ms	356–508 ms

Small grids are fastest on a single rank because each solve is tiny, and every extra rank adds MPI messages. The 10 000-bus grid gets faster with more ranks even on four shared cores; on separate machines the gain is larger. Real speed-up figures need the physical cluster.

8.5 Dynamic simulation

IEEE 39-bus with its published machine data, 15s, event at 3s, GridPACK on one rank, NS-3 on node1, control center on node2.

Event	Lowest frequency	Violation	What happened
Three-phase fault at bus 16, cleared after 0.1 s	59.70 Hz	3.6 s	Voltages down to 0.56 pu; angle spread $66^\circ \rightarrow 125^\circ$ and back; exciter overshoot to 1.16 pu; 24 control commands
Generator 32 trip (650 MW)	59.42 Hz at 9.4 s	12 s	Governors arrest the decline (59.54 Hz at 15 s); under-voltage near bus 32 remains
Line 16–17 trip	59.96 Hz	0 s	Small swing
AVR fault at gen 30 (1.12), control off	59.86 Hz	11.0 s	Exciter raises the voltage over about 2 s; it stays high
Same, control on	59.92 Hz	2.0 s	3 commands; the exciter brings the voltage back

The same fault on ranks 1, 2, and 4 yields identical voltages, frequencies, and angles. The 2000-bus grid (typical data for 432 machines, trip of the largest generator) reaches 59.81 Hz, with identical results for ranks 1 and 2.

Grid	1 rank	2 ranks (one node)	4 ranks (node1 + node2)
IEEE 39, per PMU frame	6.6 ms	12.9 ms	274 ms
ACTIVSg2000, per PMU frame	152 ms	118 ms	–

A small grid runs faster than real time on one rank (15 s simulated in about 7 s of wall time). The dynamic simulation exchanges many small messages at every integration step, so ranks spread across two containers on the shared 4-core test machine wait on each other; on separate machines, this cost is far lower, and large grids already benefit from more ranks.

9. Sizing and storage

Item	Measured size
Software per machine (/opt/cps + Ubuntu packages)	about 3 GB (plus about 10 GB for Ubuntu itself)
One IEEE 118 run, 10 s	about 1 MB
One 2000-bus run, 10 s (153 088 PMU frames)	about 14 MB
One 10 000-bus run, 10 s	about 70 MB (estimated)
One IEEE 39 dynamic run, 15 s	about 1 MB
One 2000-bus dynamic run, 5 s (true voltages every frame)	about 13 MB
An uploaded 2000-bus grid	about 2 MB

Run sizes grow linearly with duration. 1000 runs of the 2000-bus grid for 60s require about 85 GB, so 500 GB on the head (or file server) can hold several thousand experiments.

10. Limitations and next steps

Limitation	Next step
Tested on a virtual cluster (containers on one machine)	Install on separate machines with NFS; measure speed-up on 1, 2, 4 nodes
Simulated PMUs; frames are JSON, not IEEE C37.118	Connect real PMUs and a real PDC (e.g. openPDC) through C37.118
Dynamic mode: constant-impedance loads; typical machine data for grids without a .dyr file; bolted faults	Load models; upload .dyr files from the dashboard; fault impedances
PMU frames carry phasors only	Add frequency and rate-of-change-of-frequency channels (the dynamic simulation computes them)
PSS/E files are converted through MATPOWER	Read PSS/E directly; import the PGCB grid model
One administrator login; access over VPN or SSH	HTTPS and individual user accounts for many users

11. Troubleshooting

Problem	What to do
Login fails on the dashboard or editor	The user is admin; set the password again with <code>sudo cps-passwd</code>
Dashboard says "Experiment API not reachable"	<code>systemctl status cps-api</code> ; <code>journalctl -u cps-api</code>
A job fails	Read the log under the job; the federate logs are in the run directory
A node is not reachable	<code>sudo -u cps ssh node1 true</code> on the head; check the NFS mount and <code>systemctl status cps-node</code> on the node
MPI hangs between nodes	Set <code>OMPI_MCA_btl_tcp_if_include</code> and <code>OMPI_MCA_oob_tcp_if_include</code> to the cluster interface; open the firewall
A grid upload is rejected	The message names the problem (missing <code>mpc.gen</code> , no slack bus, power flow does not converge)
An experiment waits in the queue	It needs more MPI slots than are free; it starts when a running experiment ends
A dynamic run stops with "the dynamic simulation failed"	Read <code>gridpack/dsf_server.log</code> in the run directory; check the machine data (<code>gridpack/case.dyr</code>) and that the grid's power flow converges

Appendix A. Repository layout

Directory	Contents
<code>cases/</code>	IEEE 14–300 bus inputs, topology, and reference solutions; IEEE 39 machine data (<code>dynamics.dyr</code>)
<code>federates/</code>	<code>grid_fed.py</code> , <code>cc_fed.py</code> , <code>observer_fed.py</code> , <code>run_experiment.py</code> ; <code>cpslib/</code> (network model, PMU placement, state estimation, attacks, GridPACK client, uploaded grids, cluster placement)
<code>gridpack/</code>	<code>pf_server</code> (power flow) and <code>dsf_server</code> (dynamic simulation): persistent MPI GridPACK servers; <code>patches/</code> applied to GridPACK 3.5
<code>ns3-scratch/helictest/</code>	NS-3 network federate
<code>webapp/</code>	Experiment API, scenario format, and template
<code>node-red/</code>	Dashboard flows and settings
<code>legacy/</code>	Original single-PMU IEEE 14 demo used by the console
<code>tools/</code>	<code>make_cases.py</code> (MATPOWER → testbed cases)
<code>install/</code>	<code>install.sh</code> , build steps, <code>configure.py</code> , systemd units, settings example
<code>docker/</code>	Dockerfile, entrypoint, compose files
<code>documentation/</code>	This guide is for <code>installation.md</code> , <code>cluster.md</code> , user guide, and technical report of the preceding version